

Probeklausur — Musterlösung

Lösungen, Punkteverteilung und typische Fehler

100

PUNKTE IM DETAIL

4

AUFGABENBLÖCKE

17

MARKIERTE FALLEN

INHALT IN KÜRZE

TEIL 0 — Hinweise zur Musterlösung

BLOCK 1 — SQL (40 Punkte)

BLOCK 2 — Functions und Trigger (25 Punkte)

BLOCK 3 — JDBC (20 Punkte)

BLOCK 4 — Datenbankkonzepte (15 Punkte)

SO ARBEITEST DU DAMIT

- 1 **Erst selbst lösen.** Die Musterlösung nützt nur, wenn du vorher eine eigene Antwort hingeschrieben hast.
- 2 **Nach Punkteverteilung korrigieren.** Zu jeder Aufgabe steht, wofür es wie viele Punkte gibt — so siehst du, was fehlt.
- 3 **Die ⚠️-Blöcke ernst nehmen.** Dort stehen genau die Fehler, die in dieser Aufgabenart am häufigsten Punkte kosten.
- 4 **Bei SQL gilt: mehrere Wege sind richtig.** Bewertet wird das Ergebnis plus die geforderten Konstrukte.

FARBCODE DIESES LERNZETTELS

MUSS**Muss sitzen**

Kam in Klausuren mehrfach dran — sicher beherrschen.

WICHTIG: Prüfungsrelevant

Kommt vor, aber nicht in jeder Klausur.

BASIS**Grundlagen**

Verständnis-Unterbau, selten direkt gefragt.

**Merksatz**

Kernaussage — genau so formulierbar in die Klausur schreiben.

**Falle**

Typischer Fehler bzw. Punktabzug-Klassiker.

**Beispiel**

Durchgerechnete Aufgabe oder Musterlösung.

**Frage**

Selbsttest — erst antworten, dann weiterlesen.

**Originalton**

Wörtliches Zitat aus Folien oder Klausur.

**Ergänzt**

Aus Fachliteratur ergänzt, nicht wörtlich aus den Folien.

**Tipp**

Eselsbrücke oder Vorgehensstrategie.

Musterlösung zur selbst erstellten Probeklausur. Alle SQL-Lösungen sind für PostgreSQL 16 und das Schema chinook formuliert.

Inhaltsverzeichnis

TEIL 0 — Hinweise zur Musterlösung 2

BLOCK 1 — SQL (40 Punkte) 2

Aufgabe 1.1 — Tabelle anlegen	2
Aufgabe 1.2 — Integritätsbedingung nachträglich ändern ..	3
Aufgabe 1.3 — Kann eingefügt werden?	3
Aufgabe 1.4 — Ergebnis der Abfrage ermitteln	3
Aufgabe 1.5 — Multiple Choice	3
Aufgabe 1.6 — Abfragen formulieren	4

BLOCK 2 — Functions und Trigger (25 Punkte) 5

Aufgabe 2.1 — Ergebnis einer Function ermitteln	5
Aufgabe 2.2 — Function schreiben	6
Aufgabe 2.3 — Trigger und Trigger-Funktion	6

BLOCK 3 — JDBC (20 Punkte) 7

Aufgabe 3.1 — Programmrumpf ergänzen	7
Aufgabe 3.2 — SQL-Injection	7

BLOCK 4 — Datenbankkonzepte (15 Punkte) 8

Aufgabe 4.1 — ACID und BASE	8
Aufgabe 4.2 — Anomalien und Isolation Level	8
Aufgabe 4.3 — NoSQL	9

TEIL 0

Hinweise zur Musterlösung

💬 Zu jeder Aufgabe steht zuerst die **Kurzlösung**, danach die **Punkteverteilung** und — wo sinnvoll — die **typischen Fehler**, die genau diese Aufgabe in der Klausur kosten.

⚠️ Bei SQL sind fast immer mehrere Formulierungen richtig. Bewertet wird das **Ergebnis** plus die geforderten Konstrukte (z. B. „mit Unterabfrage“). Die hier gezeigte Variante ist eine mögliche Musterlösung, keine einzige richtige.

BLOCK 1 — SQL (40 Punkte)

Aufgabe 1.1 — Tabelle anlegen

8 P.

SQL

```
CREATE TABLE chinook.album (  
  album_id      int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  title         varchar(160) NOT NULL,  
  artist_id     int NOT NULL,  
  release_year  int,  
  CONSTRAINT fk_album_artist FOREIGN KEY (artist_id)  
    REFERENCES chinook.artist (artist_id)  
    ON DELETE RESTRICT,  
  CONSTRAINT chk_release_year CHECK (release_year BETWEEN 1900 AND 2100),  
  CONSTRAINT uq_album_title_artist UNIQUE (title, artist_id)  
);
```

📝 **Punkteverteilung:** PK mit Autowert 2 · title Typ + NOT NULL 1 · FK 2 · ON DELETE RESTRICT 1 · CHECK 1 · UNIQUE 1.

🔑 **GENERATED ALWAYS AS IDENTITY ist der Standard-SQL-Weg.** serial wird ebenfalls akzeptiert, ist in PostgreSQL aber veraltet.

⚠️ Typische Fehler:

- ON DELETE CASCADE statt RESTRICT — die Aufgabe verlangt ausdrücklich, dass der Künstler **nicht** gelöscht werden darf. NO ACTION ist ebenfalls korrekt (und ist der Default), CASCADE ist falsch.

- `UNIQUE (title)` allein — gefordert ist die **Kombination**.
- `CHECK` auf eine Spalte, die `NULL` sein darf: Ein `CHECK` gilt als erfüllt, wenn der Ausdruck `NULL` ergibt. `release_year IS NULL` ist damit erlaubt — genau das verlangt „sofern angegeben“.

Aufgabe 1.2 — Integritätsbedingung nachträglich ändern

4 P.

SQL

```
-- a)
ALTER TABLE chinook.customer ADD CONSTRAINT uq_customer_email UNIQUE (email);

-- b)
ALTER TABLE chinook.customer ADD COLUMN phone varchar(24);
```

 Je 2 Punkte. Ohne `CONSTRAINT <name>` gibt es keinen Abzug, der Name ist aber gute Praxis.

 `ADD CONSTRAINT ... UNIQUE` schlägt fehl, wenn bereits Duplikate in der Spalte stehen — das ist genau das Szenario aus der Übung „`UNIQUE INDEX` auf `last_name`“.

Aufgabe 1.3 — Kann eingefügt werden?

4 P.

a) **Nein**. Der Fremdschlüssel `artist_id` verweist auf `artist(artist_id)`; zu `99` existiert dort kein Datensatz. Die **referentielle Integrität** wird verletzt → `FOREIGN KEY violation`.

b) **Nein**. `1890` verletzt die `CHECK`-Bedingung `release_year BETWEEN 1900 AND 2100` → `CHECK constraint violation`.

 Je 1 Punkt für die Entscheidung, je 1 Punkt für die Begründung mit dem **Namen der verletzten Bedingung**.

 Die Begründung „geht nicht, weil der Künstler fehlt“ ist unvollständig — gefragt ist, **welche** Integritätsbedingung greift.

Aufgabe 1.4 — Ergebnis der Abfrage ermitteln

8 P.

```
billing_country |anzahl|umsatz|
-----+-----+-----+
Germany        |      3| 28.71|
Brazil         |      2| 15.84|
Canada         |      2| 10.89|
```

 **Rechenweg:**

Land	Zeilen	Summe	<code>HAVING count(*) > 1</code>
Germany	3	$8,91 + 13,86 + 5,94 = \mathbf{28,71}$	erfüllt
Canada	2	$1,98 + 8,91 = \mathbf{10,89}$	erfüllt
Brazil	2	$13,86 + 1,98 = \mathbf{15,84}$	erfüllt
USA	1	0,99	nicht erfüllt → fällt raus

 **Punkteverteilung:** korrekte Gruppen 2 · korrekte Summen 3 · `HAVING` korrekt angewendet (USA fehlt) 2 · Sortierung absteigend nach `umsatz` 1.

 **Die zwei Klassiker:** USA stehen lassen (`HAVING` übersehen) und nach `anzahl` statt nach `umsatz` sortieren. `ORDER BY umsatz DESC` bezieht sich auf den **Alias** des Aggregats.

Aufgabe 1.5 — Multiple Choice

6 P.

a) Index

- ☐ Ein Index stellt sicher, dass alle Werte einer Spalte eindeutig sind.
- ☒ Ein Index beschleunigt Such- und Abfragevorgänge.

- ☒ Ein Index verlangsamt `INSERT`, `UPDATE` und `DELETE`.
- ☐ Ein Index erzwingt die referentielle Integrität zwischen zwei Tabellen.

🔑 Eindeutigkeit erzwingt nur ein `UNIQUE INDEX` bzw. ein `UNIQUE`-Constraint — ein gewöhnlicher Index nicht. Referentielle Integrität ist Sache des **Fremdschlüssels**.

b) Views

- ☐ Eine View speichert die Ergebnisdaten physisch in der Datenbank.
- ☒ Über eine View mit `JOIN` und `DISTINCT` kann in der Regel nicht eingefügt werden.
- ☒ Views können zur Rechtevergabe auf Spalten- und Zeilenebene genutzt werden.
- ☒ Eine View wird bei jedem Zugriff neu ausgewertet.

🔑 Physisch gespeichert wird nur eine `MATERIALIZED VIEW`. Nicht aktualisierbar wird eine View durch `JOIN`, `DISTINCT`, `GROUP BY`, Aggregate oder Mengenoperationen — weil die Änderung dann nicht eindeutig auf die Originaltabellen abgebildet werden kann.

c) DELETE und TRUNCATE

- ☐ `TRUNCATE` kann mit einer `WHERE`-Bedingung eingeschränkt werden.
- ☒ `DELETE` löst Trigger aus, `TRUNCATE` in der Standardeinstellung nicht.
- ☐ Beide Befehle löschen die Tabellenstruktur mit.
- ☒ `DELETE` ohne `WHERE` löscht alle Zeilen der Tabelle.

🔑 Die Struktur löscht nur `DROP TABLE`. `TRUNCATE` kennt kein `WHERE` — es entfernt immer alle Zeilen.

📝 2 Punkte je Teilaufgabe, nur bei vollständig korrekter Ankreuzung.

Aufgabe 1.6 — Abfragen formulieren

10 P.

a) Alben von Metallica (3 P.)

SQL

```
SELECT al.title
FROM chinook.album al
JOIN chinook.artist ar ON ar.artist_id = al.artist_id
WHERE ar.name = 'Metallica';
```

📝 JOIN korrekt 2 · `WHERE` auf `artist.name` 1. Eine Unterabfrage (`WHERE artist_id = (SELECT ...)`) ist ebenfalls voll korrekt.

b) Tracks je Genre (4 P.)

SQL

```
SELECT g.name,
       count(*) AS anzahl,
       round(avg(t.milliseconds) / 1000.0, 1) AS schnitt_sekunden
FROM chinook.track t
JOIN chinook.genre g ON g.genre_id = t.genre_id
GROUP BY g.name
HAVING count(*) >= 10
ORDER BY anzahl DESC;
```

📝 `GROUP BY` 1 · Umrechnung ms → s 1 · `round(..., 1)` 1 · `HAVING` + Sortierung 1.

⚠ **Wo hier wirklich die Falle liegt:** `avg()` über eine **Integer-Spalte** liefert in PostgreSQL bereits `numeric` — `track.milliseconds` ist `int`, also ist `avg(t.milliseconds)/1000` schon eine `numeric`-Division und `round(..., 1)` funktioniert auch ohne das `.0`. Es schadet aber nichts. Gefährlich wird es an zwei anderen Stellen: bei einer **Division pro Zeile** (`t.milliseconds/1000` ist Integer-Division und schneidet ab — dort `1000.0` schreiben) und bei `avg()` über `double precision`, denn `round(double precision, int)` gibt es in PostgreSQL nicht — dort braucht es `::numeric`.

c) Kunden ohne Rechnung (3 P.)

SQL

```
SELECT c.customer_id, c.first_name, c.last_name
FROM   chinook.customer c
WHERE  NOT EXISTS (SELECT 1
                   FROM   chinook.invoice i
                   WHERE  i.customer_id = c.customer_id);
```

📝 Unterabfrage 2 · korrekte Korrelation bzw. korrektes `NOT IN 1`.

⚠ **Die Falle bei `NOT IN`:** `WHERE customer_id NOT IN (SELECT customer_id FROM invoice)` liefert **gar keine Zeile**, sobald die Unterabfrage auch nur ein `NULL` enthält — denn `x NOT IN (... , NULL)` ergibt `UNKNOWN`. `NOT EXISTS` oder ein `LEFT JOIN ... WHERE i.invoice_id IS NULL` sind robust.

BLOCK 2 — Functions und Trigger (25 Punkte)

Aufgabe 2.1 — Ergebnis einer Function ermitteln

5 P.

```
gesamtanzahl|im_bereich|
-----+-----+
            8|            3|
```

📝 **Rechenweg für `anzahl(2, 9)`** — gezählt wird `total >= 2 AND total <= 9`:

total	8,91	13,86	1,98	5,94	13,86	8,91	0,99	1,98
im Bereich?	✓	✗	✗	✓	✗	✓	✗	✗

→ **3 Rechnungen.** `count(*)` über `invoice` liefert unabhängig davon **8**.

📝 **Begründung (2 P.):** `chinook.anzahl(2, 9)` ist eine **skalare Funktion ohne Bezug zur aktuellen Zeile**. Sie wird einmal ausgewertet und steht in jeder Ergebniszeile mit demselben Wert; `count(*)` zählt dagegen die Zeilen der äußeren Abfrage. Die beiden Werte sind voneinander unabhängig.

⚠ **`BETWEEN` bzw. `>= ... AND <= ...` ist **inklusive** beider Grenzen** — aber 1,98 und 0,99 liegen unterhalb von 2 und fallen trotzdem heraus. Genau solche Randwerte sind der Grund, warum die Aufgabe „Ergebnis ermitteln“ heißt und nicht „Abfrage lesen“.

📝 Ergebniswerte 3 · Begründung 2.

Aufgabe 2.2 — Function schreiben

8 P.

SQL

```
CREATE OR REPLACE FUNCTION chinook.umsatz_kunde(p_customer_id int)
RETURNS numeric
LANGUAGE plpgsql
AS $$
DECLARE
    v_summe numeric;
BEGIN
    IF NOT EXISTS (SELECT 1 FROM chinook.customer
                   WHERE customer_id = p_customer_id) THEN
        RAISE EXCEPTION 'Kunde % existiert nicht', p_customer_id;
    END IF;

    SELECT coalesce(sum(total), 0)
    INTO v_summe
    FROM chinook.invoice
    WHERE customer_id = p_customer_id;

    RETURN v_summe;
END;
$$;
```

 Signatur + RETURNS numeric 1 · LANGUAGE plpgsql mit DECLARE/BEGIN/END 2 · SELECT ... INTO 1 · coalesce(..., 0) 2 · RAISE EXCEPTION 2.

 sum() über eine leere Menge liefert NULL, nicht 0. coalesce ist deshalb kein Schönheitsfehler, sondern die halbe Aufgabe.

 In LANGUAGE plpgsql ist RETURN zwingend — anders als bei LANGUAGE sql, wo der Rückgabewert implizit die letzte Anweisung ist.

Aufgabe 2.3 — Trigger und Trigger-Funktion

12 P.

a) Trigger-Funktion (7 P.)

SQL

```
CREATE OR REPLACE FUNCTION chinook.trg_genre_name_default()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.name := btrim(NEW.name);

    IF NEW.name IS NULL OR NEW.name = '' THEN
        NEW.name := 'Untitled Genre';
    END IF;

    RETURN NEW;
END;
$$;
```

b) Trigger (3 P.)

SQL

```
CREATE TRIGGER genre_name_default
BEFORE INSERT ON chinook.genre
FOR EACH ROW
EXECUTE FUNCTION chinook.trg_genre_name_default();
```

c) Warum BEFORE ? (2 P.)

Nur in einem BEFORE -Trigger kann der Datensatz über NEW noch verändert werden, bevor er geschrieben wird. Ein AFTER -Trigger läuft erst nach dem Schreiben; Änderungen an NEW haben dort keine Wirkung mehr und man müsste ein zusätzliches UPDATE absetzen.

 RETURNS trigger 1 · NEW korrekt verwendet 2 · Prüfung auf NULL und Leerstring 2 · btrim 1 · RETURN NEW 1 · Trigger mit BEFORE INSERT ... FOR EACH ROW 3 · Begründung 2.

Drei Klassiker:

- RETURN NULL statt RETURN NEW in einem BEFORE -Row-Trigger — damit wird das INSERT stillschweigend abgebrochen.
- RETURNS trigger vergessen.
- EXECUTE PROCEDURE ist noch erlaubt, aber veraltet; korrekt ist EXECUTE FUNCTION .

BLOCK 3 — JDBC (20 Punkte)

Aufgabe 3.1 — Programmrumpf ergänzen

12 P.

JAVA

```
// a) Kunden eines Landes ausgeben
String sqlSelect = "SELECT last_name, first_name FROM customer WHERE country = ?";
PreparedStatement ps = conn.prepareStatement(sqlSelect);
ps.setString(1, land);
ResultSet rs = ps.executeQuery();

while (rs.next()) {
    System.out.println(rs.getString("last_name") + ", " + rs.getString("first_name"));
}
rs.close();
ps.close();

// b) company auf NULL setzen
String sqlUpdate = "UPDATE customer SET company = NULL WHERE customer_id = ?";
PreparedStatement psUp = conn.prepareStatement(sqlUpdate);
psUp.setInt(1, 5);
int geaendert = psUp.executeUpdate();
System.out.println("Geänderte Zeilen: " + geaendert);
psUp.close();
```

 a) preparedStatement mit ? 2 · setString 1 · executeQuery 1 · while (rs.next()) 2 · Spaltenzugriff 1. b) executeUpdate statt executeQuery 2 · Rückgabewert als Zeilenzahl ausgeben 2 · setInt 1.

 Die Unterscheidung ist Punktegold: executeQuery liefert ein ResultSet, executeUpdate liefert ein int mit der Anzahl betroffener Zeilen. Für DDL/DML ohne Ergebnismenge ist executeUpdate richtig.

 rs.next() vor dem ersten Zugriff aufrufen — der Cursor steht anfangs vor der ersten Zeile. Ein rs.getString(...) ohne vorheriges next() wirft eine SQLException .

Aufgabe 3.2 — SQL-Injection

8 P.

a) Angriff (3 P.)

Eingabe: ' OR '1'='1

Daraus entsteht:

SQL

```
SELECT * FROM customer WHERE last_name = '' OR '1'='1'
```

Die Bedingung ist für **jede** Zeile wahr → die gesamte Tabelle wird zurückgegeben.

b) Absicherung (3 P.)

JAVA

```
String sql = "SELECT * FROM customer WHERE last_name = ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, eingabe);
ResultSet rs = ps.executeQuery();
```

c) Warum das schützt (2 P.)

Beim `PreparedStatement` wird die **SQL-Anweisung getrennt von den Daten** an die Datenbank übergeben und dort **vorab geparkt**. Der Platzhalter `?` ist bereits als Wert festgelegt, bevor die Eingabe eintrifft — die Eingabe kann die Struktur der Anweisung deshalb nicht mehr verändern, sondern wird immer als **reiner Zeichenkettenwert** behandelt.



„Ich escape die Hochkommata“ ist **keine** vollwertige Antwort: Escaping ist fehleranfällig, datenbankspezifisch und schützt nicht bei numerischen oder Bezeichner-Kontexten.

BLOCK 4 — Datenbankkonzepte (15 Punkte)

Aufgabe 4.1 — ACID und BASE

6 P.

a) ACID (4 P., je 1)

A	Atomicity — Atomarität	Eine Transaktion wird ganz oder gar nicht ausgeführt. Bei einem Fehler wird vollständig zurückgerollt.
C	Consistency — Konsistenz	Die Datenbank geht von einem konsistenten Zustand in einen konsistenten Zustand über; alle Integritätsbedingungen gelten vorher wie nachher.
I	Isolation — Isolation	Nebenläufige Transaktionen beeinflussen sich nicht gegenseitig ; jede läuft, als wäre sie allein.
D	Durability — Dauerhaftigkeit	Nach dem <code>COMMIT</code> sind die Änderungen dauerhaft gespeichert, auch bei Systemabsturz.

b) BASE (2 P.)

Basically Available, Soft state, Eventually consistent.



Der grundlegende Unterschied: ACID garantiert Konsistenz zu **jedem** Zeitpunkt, notfalls auf Kosten der Verfügbarkeit. BASE gibt die sofortige Konsistenz auf und garantiert nur, dass das System **irgendwann** konsistent wird (*eventual consistency*) — dafür bleibt es jederzeit verfügbar und gut horizontal skalierbar. Typisch für verteilte NoSQL-Systeme.

Aufgabe 4.2 — Anomalien und Isolation Level

5 P.

a) (2 P.) Phantom Read (Phantomlesen). Zwischen den beiden Lesevorgängen von T1 fügt T2 eine **neue Zeile** ein und committet; die zweite Abfrage sieht dadurch einen Datensatz, den es vorher nicht gab.



Nicht mit **Non-Repeatable Read** verwechseln: dort wird eine **bestehende** Zeile geändert oder gelöscht. Phantom = **neue Zeilen** in derselben Suchbedingung.

b) (3 P.) Verhindert wird das Phantomlesen erst durch `SERIALIZABLE`. Voreinstellung in PostgreSQL ist `READ COMMITTED`.

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
<code>READ UNCOMMITTED</code>	möglich ¹	möglich	möglich
<code>READ COMMITTED</code> ← Default	—	möglich	möglich
<code>REPEATABLE READ</code>	—	—	möglich ²
<code>SERIALIZABLE</code>	—	—	—



¹ PostgreSQL kennt `READ UNCOMMITTED` syntaktisch, behandelt es aber wie `READ COMMITTED` — Dirty Reads treten dort nie auf.



² Nach SQL-Standard sind Phantome in `REPEATABLE READ` möglich; PostgreSQLs Implementierung über Snapshots verhindert sie faktisch bereits.

Aufgabe 4.3 — NoSQL

4 P.

a) (2 P.)

Kategorie	Vertreter	Datenmodell
Key-Value Store	Redis, Riak	Schlüssel → beliebiger Wert
Dokumentenorientiert	MongoDB, CouchDB	JSON/BSON-Dokumente
Spaltenorientiert (Wide Column)	Cassandra, HBase	Spaltenfamilien
Graphdatenbank	Neo4j	Knoten und Kanten

b) (2 P.) Consistency, Availability, Partition tolerance.



Ein klassisches, **nicht verteiltes** RDBMS ist ein **CA**-System: es garantiert Konsistenz und Verfügbarkeit, ist aber nicht auf Partitionstoleranz ausgelegt. Sobald ein System verteilt ist, ist **P** unverzichtbar — dann muss zwischen **CP** (z. B. HBase, MongoDB in der Standardkonfiguration) und **AP** (z. B. Cassandra, Riak) gewählt werden.